Cyber-Physical Systems — Summer Term 2026

Prof. Merten Stender, TU Berlin

P13: Web App for Non-Expert Users

Importance and Role in the Overall Project

A CPS system that can only be understood by reading raw data messages is not usable by the person who owns the balcony. This project bridges the gap between the technical system and the human who depends on it. Someone who wants to know whether the plants are being looked after, not how many milliseconds the controller's state machine spent in the "watering" state. Good human-CPS interfaces are genuinely difficult to design: you must decide which information matters, how to present uncertainty, when to alert versus when to stay quiet, and how to enable manual intervention without making accidental actions too easy. This project is also the only one with an explicit user-facing concern, making it home to one of the important cross-cutting themes of the course: how do humans interact with autonomous physical systems, and how should those systems communicate their state to non-expert users?

Core Disciplines Involved

This project combines web development, UI/UX design, and API integration.

- **Web development**: building a functional frontend and a supporting backend that connects to the rest of the system; making the interface accessible from a phone or browser without installation
- **UI/UX design**: information hierarchy, visual clarity, designing for users who have no technical background in CPS, and avoiding information overload
- **API design**: defining how the web app retrieves data from the logging system and sends commands to the controller; this interface must be clean and versioned
- **Access control**: deciding who can see status information and who is allowed to issue commands; read-only access and control access should be separated

---

Interfaces to the Global System

MUST — these are required for this project to fulfil its purpose:

- Display the current system status in a form understandable to a non-technical user: is the system operating normally, when did it last water, and what is the current soil moisture?
- Display active alerts from the anomaly detector: these should be prominent and clearly explained in plain language
- Provide a manual watering trigger that sends a command to the controller via the bus; this override must include a confirmation step to prevent accidental activation

CAN — valuable additions that improve usefulness:

- Show historical trend graphs for soil moisture and watering events, drawn from the logging system's data store
- Display tank fill level and time-to-empty estimate from the tank monitoring project
- Allow editing of basic watering parameters (e.g. target moisture level, watering schedule) without requiring direct access to the Pi
- Implement push notifications or email alerts for critical events, so users do not need to actively check the app

---

Minimum Expected Outcome

This baseline represents a usable minimum product.

- A web page accessible from a browser on the local network showing current sensor readings and system status, updated at a regular interval
- Active anomaly alerts are displayed prominently and described in plain language, not as raw error codes
- A manual watering button exists, is protected by a confirmation step, and successfully sends a command to the bus when activated

---

Expected Outcome

The minimum, plus the following to make the project solid and educationally complete.

- A clean, mobile-friendly interface that requires no technical knowledge to interpret; someone unfamiliar with the project should be able to understand the system state within 30 seconds of opening the page
- Historical trend graphs for at least soil moisture and watering events, covering the past 24 hours as a default view
- An alert panel showing current and recent anomalies with plain-language descriptions and timestamps
- Basic access control: the app requires authentication to access, and the ability to issue commands requires a separate, higher-privilege role
- A usability reflection in the report: what design decisions were made to make the interface accessible, and how were they evaluated?

Stretch goal: a notification mechanism that pushes alerts to the user's phone or email when something critical happens, without requiring the user to actively monitor the web page.

Where to Start

- **Human-computer interaction principles**: read about information hierarchy, progressive disclosure, and the difference between status information and alert information; think carefully about what a non-technical user actually needs to know versus what is just interesting to engineers
- **Web application architecture**: understand the basic structure of a web app: a frontend (what the user sees in the browser), a backend (a server that provides data), and how they communicate; understand when a simple static page with periodic data fetching is sufficient versus when live push updates are needed

- **Connecting to the message bus from a web backend**: think about how your backend server will receive data from the bus and serve it to the browser; these are two separate data flows that need to be handled separately
- **Access control in web applications**: study the difference between authentication (proving who you are) and authorisation (what you are allowed to do); understand why issuing physical commands must be protected more carefully than reading sensor data
- **Existing IoT dashboards for inspiration and critique**: look at smart home apps, weather station displays, and industrial monitoring tools; evaluate critically what works for a non-expert user and what creates unnecessary complexity